

Tropical light exposure & health

Johannes Zauner
Technical University of Munich, Germany

2026-06-01

Table of contents

Preface	1
How this page works	3
Installation	3
Load and visualize glasses-level light exposure data for Costa Rica	5
Calculate common exposure metrics	8
Load and merge sleep-wake data with light exposure data	10
Adherence to Brown et al. recommendations	13
Session info	18

Preface

i About this document

This document is part of a seminar on *Open science and FAIR data practices for personal light exposure research* and demonstrates the value and necessity of open science practices to understand scientific results and increase value of collected data beyond its initial use.

Personal light exposure (PLE) varies strongly between geographic locations, photoperiod, climate, the built environment, culture, and especially dependent on human behaviour. This is important, as PLE is increasingly indicated in not just acute effects, like alertness, mood, and wellbeing, but also longterm mental, metabolic, and cardiovascular health. To support longterm health, recommendations for healthy daytime, evening, and nighttime light have been developed, based on laboratory studies on the so-called non-visual effects of light throughout this century¹.

Wearable light loggers are used to assess personal light exposure under naturalistic conditions. However, our understanding of PLE is dominated from western, industrialized, high-income countries, and especially limited to how PLE varies in different climates. The MeLiDos project captured annotated, high-resolution and multi-country datasets with a harmonized protocol in Sweden, the Netherlands, Germany, Spain, Turkey, Costa Rica, and Ghana (Figure 1).

¹Brown et al. (2025)

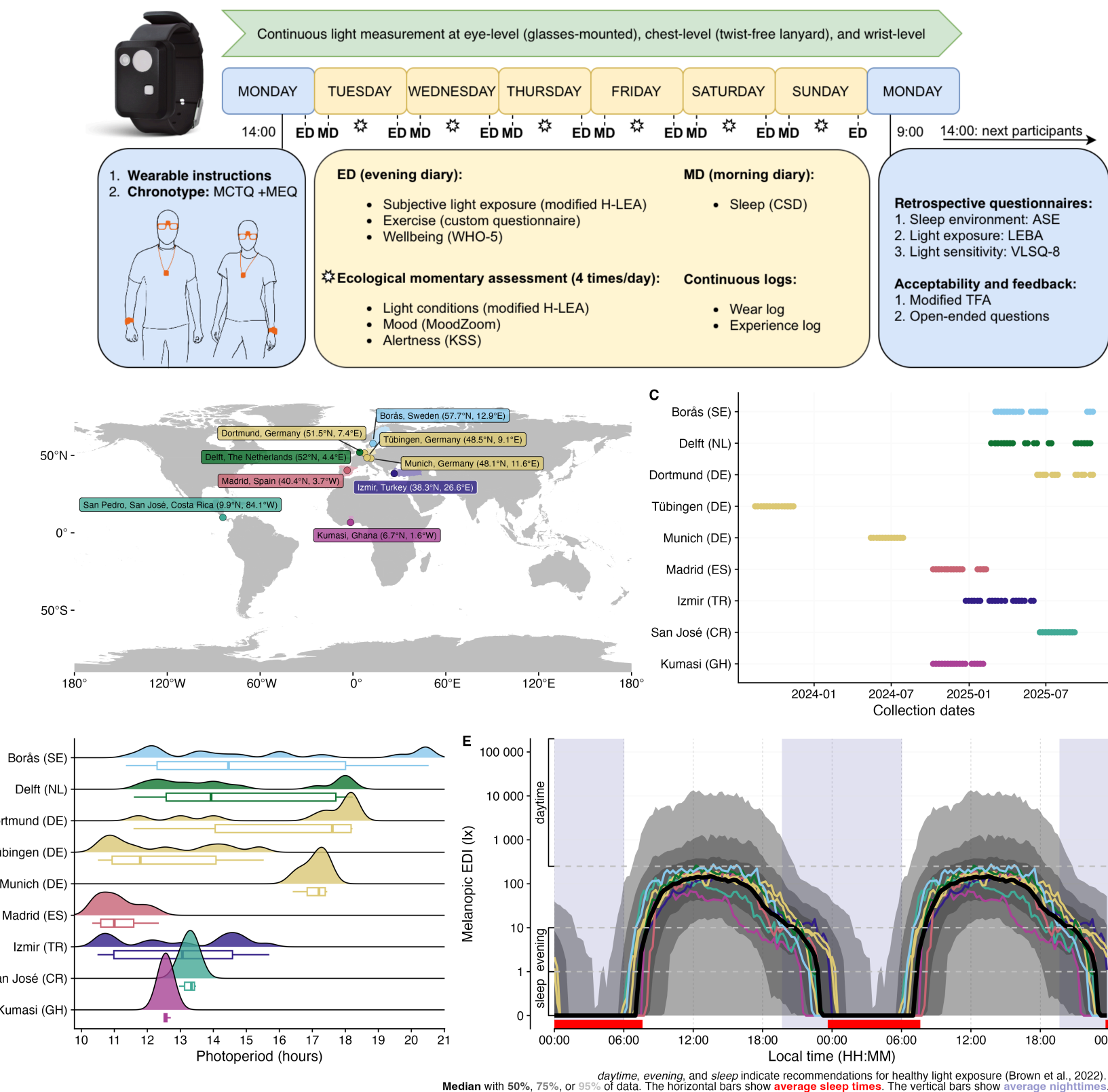


Figure 1: Overview of the MeLiDos project field study

This document uses the `melidosData` R package to load and analyze MeLiDos study data for the Costa Rica site. The document has the following goals:

- load wearable data for Costa Rica
- create plots to gain an understanding of exposure patterns
- calculate common exposure metrics
- load sleep-wake data for the same dataset
- merge sleep-wake data with PLE data
- calculate, summarize, and visualize adherence to recommendations for PLE

The analysis uses standardized processing pipelines through the `LightLogR` package. `LightLogR` is designed to facilitate the principled import, processing, and visualization of such wearable-derived data. Full documentation of `LightLogR`'s features is available on the documentation page, including numerous tutorials.

This document assumes general familiarity with the R statistical software, ideally in a data-science context².

How this page works

This document contains the script for the online course series as a Quarto script, which can be executed on a local installation of R. Please ensure that all libraries are installed prior to running the script:

```
renv::restore()
```

Installation

`melidosData` is hosted on CRAN, which means it can easily be installed from any R console through the following command:

```
install.packages("melidosData")
```

After installation, it becomes available for the current session by loading the package. We also require a number of packages. Most are automatically downloaded with `LightLogR`, but need to be loaded separately. Some might have to be installed separately on your local machine.

```
library(melidosData) #load the package
library(LightLogR) #load the package
library(tidyverse) #a package for tidy data science
library(gt) #a package for great tables
#the following packages are needed for preview functions:
library(svglite)
```

²If you are new to the R language or want a great introduction to R for data science, we can recommend the free online book *R for Data Science* (second edition) by Hadley Wickham, Mine Cetinkaya-Rundel, and Garrett Grolemund.

```
library(xml2)
library(downlit)
```

We start by making a decision on the site we want to look at and collect some metadata about it.

```
params$site
```

```
[1] "UCR"
```

```
params$dataset
```

```
[1] "glasses"
```

```
site <- params$site
dataset <- paste0("light_", params$dataset, "_lminute")
melidos_coordinates[[site]]
```

```
[1] 9.9372 -84.0509
```

```
melidos_colors[[site]]
```

```
[1] "#44AA99"
```

```
melidos_cities[[site]]
```

```
[1] "San Pedro, San José"
```

```
melidos_countries[[site]]
```

```
[1] "Costa Rica"
```

```
melidos_tzs[[site]]
```

```
[1] "America/Costa_Rica"
```

Want to use a different site? Just switch the Institution name in the code cell above

Institution (site Abbr.)	City	Country	Repository	DOI
KNUST	Kumasi	Ghana	AkuffoEtAl_Dataset_2025	10.5281/zenodo.15576731
UCR	San José	Costa Rica	Sancho-SalasEtAl_Dataset_2025	10.5281/zenodo.17289456
IZTECH	Izmir	Turkey	DidikogluEtAl_Dataset_2025	10.5281/zenodo.16568109
FUSPCEU	Madrid	Spain	BaezaEtAl_Dataset_2025	10.5281/zenodo.16834951
TUM	Munich	Germany	HildenEtAl_Dataset_2025	10.5281/zenodo.16893901
MPI	Tübingen	Germany	GuldolinEtAl_Dataset_2025	10.5281/zenodo.16895188
BAUA	Dortmund	Germany	BroszioEtAl_Dataset_2025	10.5281/zenodo.18111232
THUAS	Delft	The Netherlands	AertsEtAl_Dataset_2025	10.5281/zenodo.17979893
RISE	Borås	Sweden	NilssonTengelinEtAl_Dataset_2025	10.5281/zenodo.18925834

Table 1: Overview of the available sites in the package

Load and visualize glasses-level light exposure data for Costa Rica

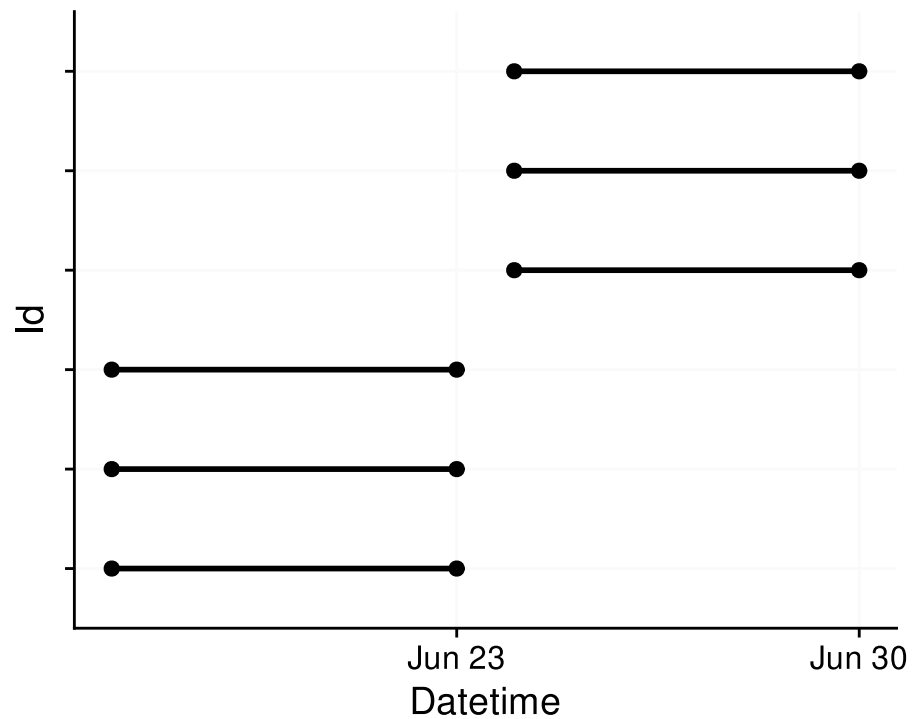
The `load_data()` function loads pre-processed data from the *MeLiDos* project. The `site` argument can be set to one or multiple sites. In our case, UCR loads the right data. Check above to see details about the site. To reduce data complexity, we use 1-minute aggregated data, which has also been pre-processed and cleaned.

```
data <- load_data(dataset, site = site)
names(data)
```

```
[1] "Id"           "Datetime"    "EVENT"       "TEMPERATURE" "ORIENTATION"
[6] "PIM"         "PIMn"        "TAT"         "TATn"        "ZCM"
[11] "ZCMn"        "LIGHT"       "IR.LIGHT"    "CAP_SENS_1"  "CAP_SENS_2"
[16] "F1"          "F2"          "F3"          "F4"          "F5"
[21] "F6"          "F7"          "F8"          "MEDI"        "CLEAR"
[26] "STATE"       "file.name"   "position"    "is.implicit"
```

We can explore this dataset in several, low-effort ways.

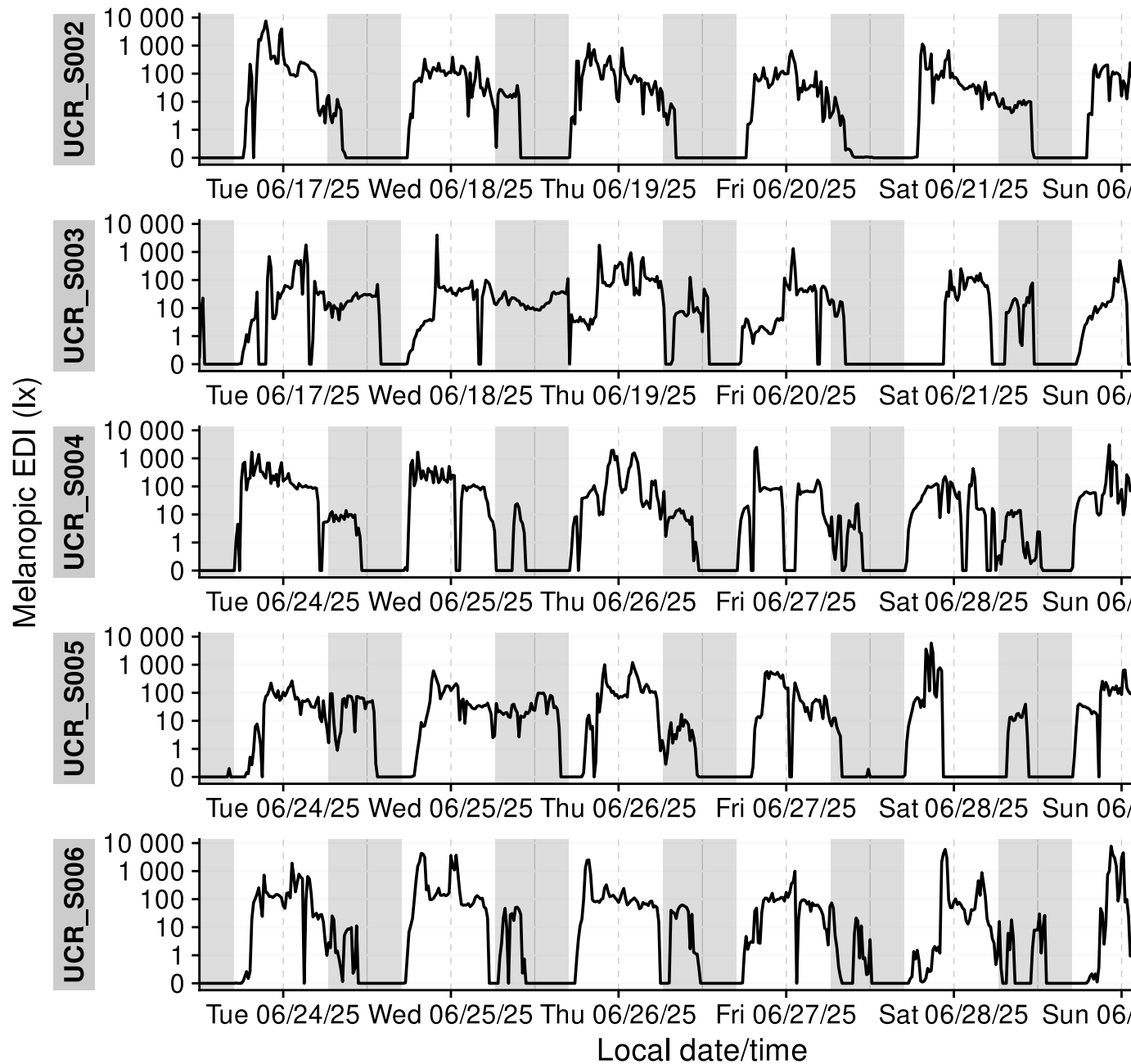
```
data |>
  gg_overview() + #create the overview plot
  theme_sub_axis_y(text = element_blank()) #remove y-axis text
```



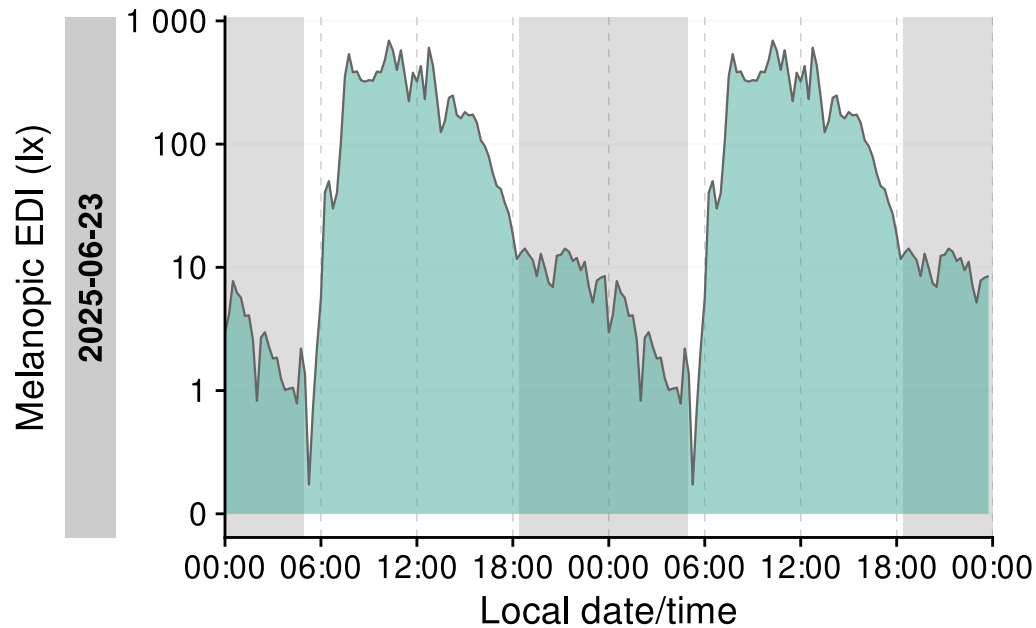
```
data |>
  summary_overview() |> #calculate overview stats
  gt() |> sub_missing() |> #show as table
  tab_header(
    paste0("Dataset overview for ",
           melidos_cities[[site]], ", ",
           melidos_countries[[site]])
  )
```

Dataset overview for San Pedro, San José, Costa Rica			
name	mean	min	max
Participants	6	—	—
Participant-days	36	6	6
Days ≥80% complete	36	6	6
Missing/Irregular	0	0	0

```
data |>
  sample_groups(5) |> #select 5 groups (participants)
  aggregate_Datetime("15 mins", type = "floor") |> #condense data to 15-minute
  intervals
  gg_days() |> #create timeline plot
  gg_photoperiod(melidos_coordinates[[site]]) #add photoperiod information
```



```
data |>
  ungroup() |> #remove by-participant grouping
  aggregate_Date(unit = "15 mins") |> #condense data to 1 day of 15-minute
  intervals
  gg_doubleplot(fill = melidos_colors[[site]]) |> #create double plot
  gg_photoperiod(melidos_coordinates[[site]]) #add photoperiod information
```



Calculate common exposure metrics

LightLogR has a summary function that calculates many common metrics and shows how they are distributed within the dataset.

```
data |>
  summary_table( #summary table function
    melidos_coordinates[[site]], #provide coordinates for photoperiod calculation
    location = melidos_cities[[site]], #provide a label for location
    site = melidos_countries[[site]], #provide a label for site
    color = melidos_colors[[site]] #provide a color for histogram generation
  )
```


Summary table				
San Pedro, San José, Costa Rica, 9.9°N, 84.1°W, TZ: America/ Costa_Rica				
Overview				
Participants	Participants	6		
Participant-days	Participant-days	36 (6 - 6)		
Days ≥80% complete	Days ≥80% complete	36 (6 - 6)		
Missing/irregular	Missing/Irregular	0.0% (0.0% - 0.0%)		
Photoperiod	Photoperiod	13h 28m (13h 27m - 13h 28m)		¹
Metrics ²				
Dose	D (lx·h)	3,140 ±2,439 (643 - 8,572)		
Duration above 250 lx	TAT250	1h 47m ±1h 24m (16m - 4h 56m)		
Duration within 1-10 lx	TWT1-10	3h 15m ±1h 32m (47m - 8h 33m)		
Duration below 1 lx	TBT1	9h 53m ±2h 40m (1h 47m - 16h 38m)		
Period above 250 lx	PAT250	31m 46s ±21m 52s (3m - 1h 29m)		
Duration above 1000 lx	TAT1000	36m 5s ±36m 53s (0s - 1h 58m)		
First timing above 250 lx	FLiT250	08:33 ±01:35 (06:06 - 11:57)		¹
Mean timing above 250 lx	MLiT250	11:23 ±01:38 (08:27 - 14:04)		¹
Last timing above 250 lx	LLiT250	15:20 ±03:05 (10:21 - 23:51)		¹
Brightest 10h midpoint	M10midpoint	12:40 ±02:07 (05:22 - 18:47)		¹
Darkest 5h midpoint	L5midpoint	03:17 ±03:07 (02:29 - 21:04)		¹
Brightest 10h mean ³	M10mean (lx)	71.2 ±56.2 (3.1 - 193.6)		
Darkest 5h mean ³	L5mean (lx)	0.1 ±0.4 (0.0 - 2.1)		
Interdaily stability	IS	0.302 ±0.087 (0.207 - 0.431)		
Intradaily variability	IV	1.379 ±0.140 (1.155 - 1.580)		
values show: mean ±sd (min - max) and are all based on measurements of melanopic EDI (lx)				

¹ Melanopic EDI limits are calculated from a participant's 24-hour light exposure (n=36) with the exception of IV and IS, which were calculated as a by-participant basis (n=6).

Load and merge sleep-wake data with light exposure data

We start by loading sleepdiaries data. Because we only want to check for data when devices were worn, we also load the wearlog information.

```
sleepdata <- load_data("sleepdiaries", site = site)
wearlog <- load_data("wearlog", site = site)
```

We can quickly check what information is available in both datasets with the `extract_labels()` function.

```
sleepdata |> extract_labels() |> head()
```

```
              Id
              "Record ID"
              bedtime
              "What time did you get into bed?"
              sleepprep
              "What time did you try to go to sleep?"
              wake
              "What time was your final awakening? i.,e. when did you wake up today?"
              out_ofbed
              "What time did you get out of bed for the day?"
              sleepdelay
              "How long did it take you to fall asleep? Please answer in minutes"
```

```
wearlog |> extract_labels() |> head()
```

```
              Id              start
              "Record ID"      "Start datetime of the state"
              state            duration
              "Wearlog state"   "Duration of the state"
              end              wearlog_event
              "End datetime of the state" "Description of the starting event"
```

In the next step, we prepare the sleepdiary data by selecting a subset containing the participant ID, as well as the time when participants prepared to sleep (`sleepprep`) and when they woke (`wake`). Because we are not only interested in labelling sleep periods, but also the in-between wake periods, we pivot the data to a longer form and transform them to intervals. Based on those sleep and wake intervals, we assign states according to Brown et al. (day, evening, night).

```
sleepdata <-
sleepdata |>
  select(Id, sleep = sleepprep, wake) |> #subset of the sleepdiaries
```

```

group_by(Id) |> #group by participant
pivot_longer(-Id, names_to = "sleep", values_to = "Datetime") |> #reshape to
one row per state
sc2interval(Statechange.colname = sleep, starting.state = "wake") |> #intervals
(with max length) instead of timestamps
sleep_int2Brown(sleep.state = "sleep", Brown.day = "wake", #Brown et al.
intervals
                Brown.evening = "pre-sleep", Brown.night = "sleep") |> #Brown
et al. intervals
mutate(sleep = case_when(is.na(sleep) & State.Brown == "pre-sleep" ~ "wake",
#fill in values for pre-sleep
                        .default = sleep))

```

Adding missing grouping variables: `Id`

```
head(sleepdata)
```

```

# A tibble: 6 × 4
# Groups:   Id [1]
  Id      sleep Interval          State.Brown
<chr>    <chr> <Interval>          <chr>
1 UCR_S001 wake  2025-06-16 00:00:00 CST--2025-06-16 18:30:00 CST wake
2 UCR_S001 wake  2025-06-16 18:30:00 CST--2025-06-16 21:30:00 CST pre-sleep
3 UCR_S001 sleep 2025-06-16 21:30:00 CST--2025-06-17 06:36:00 CST sleep
4 UCR_S001 wake  2025-06-17 06:36:00 CST--2025-06-17 18:30:00 CST wake
5 UCR_S001 wake  2025-06-17 18:30:00 CST--2025-06-17 21:30:00 CST pre-sleep
6 UCR_S001 sleep 2025-06-17 21:30:00 CST--2025-06-18 06:30:00 CST sleep

```

The transformed sleep data, as well as photoperiod information and wear states get added to the light exposure data.

```

data <-
data |>
  select(Id, Datetime, MEDI) |> #subset of light data
  add_photoperiod(melidos_coordinates[[site]]) |> #add photoperiod information
  add_states(sleepdata, start = Interval, end = Interval) |> #add sleep
information
  add_states(wearlog |> select(Id, start, end, wear = state)) #add wear
information
names(data)

```

```

[1] "Id"          "Datetime"    "MEDI"
[4] "dawn"        "dusk"        "photoperiod"

```

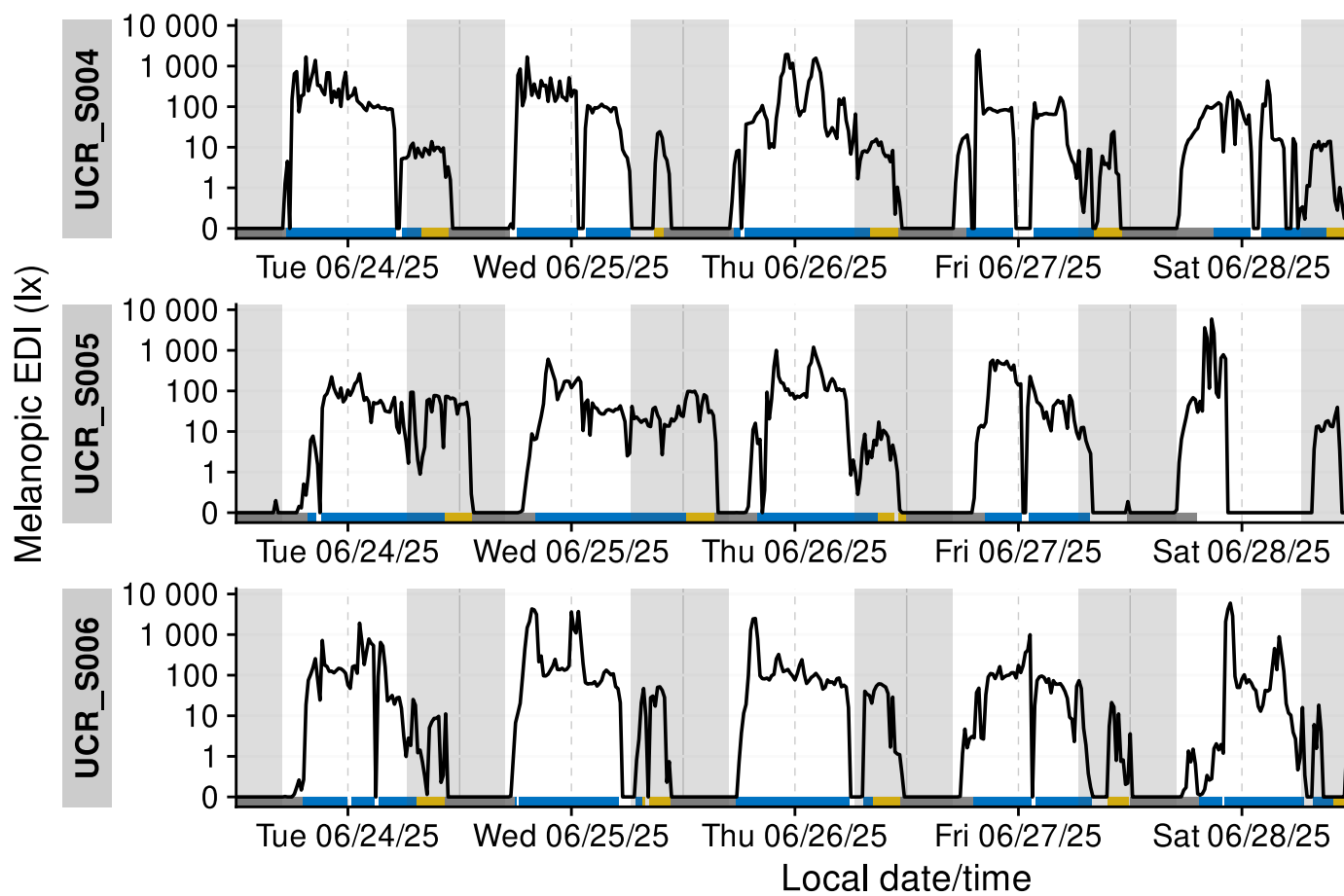
```
[7] "photoperiod.state" "sleep"          "State.Brown"
[10] "wear"
```

Next, we want to remove instances from the Brown states when the device was not worn during the day or evening.

```
#Remove non-wear data during wake or pre-sleep
data <-
data |>
  mutate(
    State.Brown = replace_when(
      State.Brown,
      wear == "off" & sleep != "sleep" ~ NA
    )
  )
```

We can visualize this combined dataset by stacking several of the previous functions and adding the state information on top.

```
data |>
  sample_groups(3) |> #select three participants
  aggregate_Datetime("15 mins", type = "floor") |> #aggregate to 15-minute bins
  mutate(State.Brown = #order factor labels (for coloring)
    factor(State.Brown, levels = c("wake", "pre-sleep", "sleep"))) |>
  gg_days() |> #create base-plot
  gg_photoperiod() |> #add photoperiod information
  gg_states(State.Brown, #add state information
    aes_fill = State.Brown, #fill by state
    ymax = 0, alpha = 1 #only create a small band
  ) +
  labs(fill = "State") # adjust legend label
```



Adherence to Brown et al. recommendations

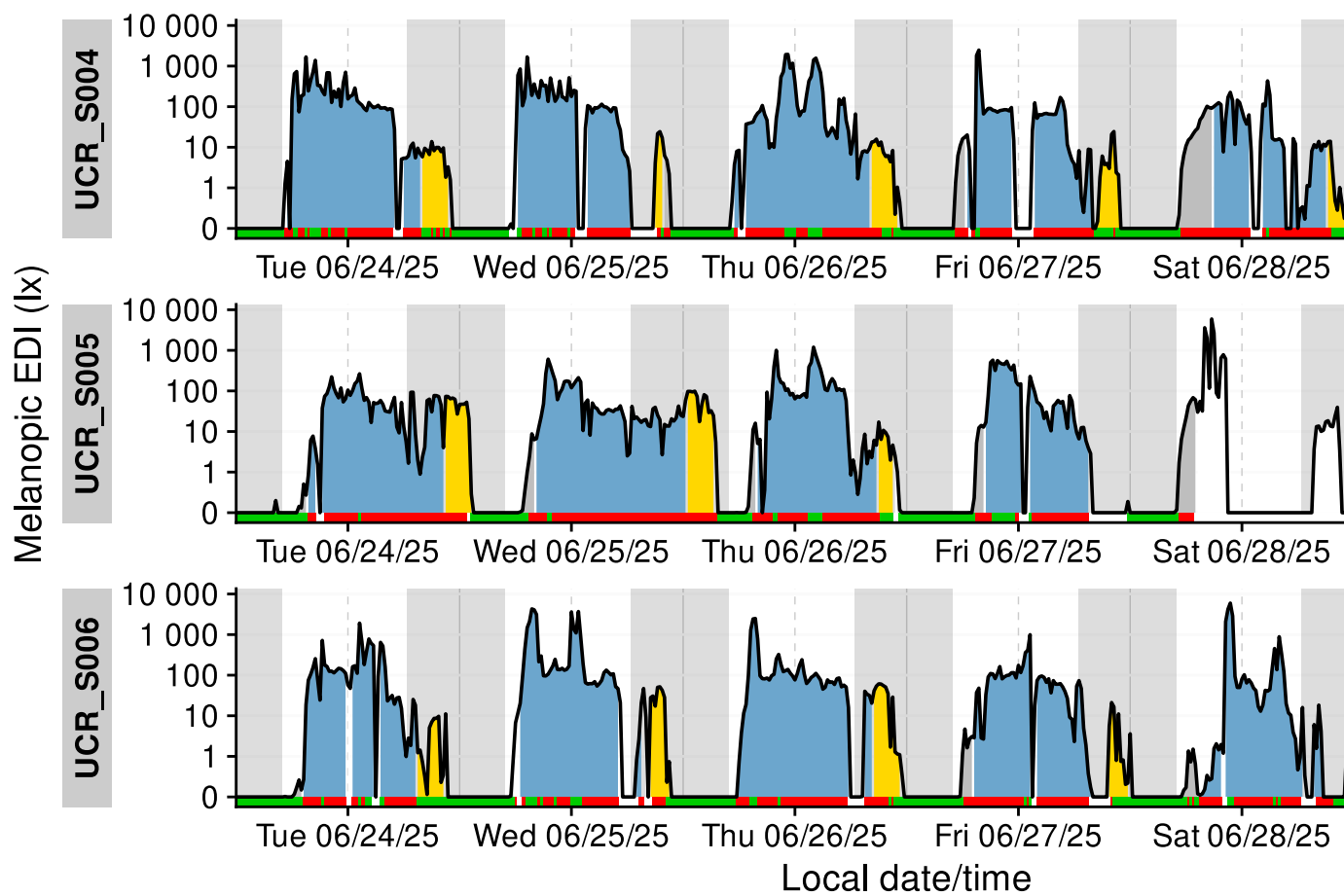
The first step is to check whether the melanopic EDI were satisfactory at a given moment through the `Brown2reference()` function.

```
data <-
data |>
  Brown2reference(Brown.day = "wake", #check whether melEDI are ok
                  Brown.evening = "pre-sleep",
                  Brown.night = "sleep")
names(data)
```

```
[1] "Id"                "Datetime"        "MEDI"
[4] "dawn"              "dusk"            "photoperiod"
[7] "photoperiod.state" "sleep"            "State.Brown"
[10] "wear"              "Reference"        "Reference.check"
[13] "Reference.difference" "Reference.label"
```

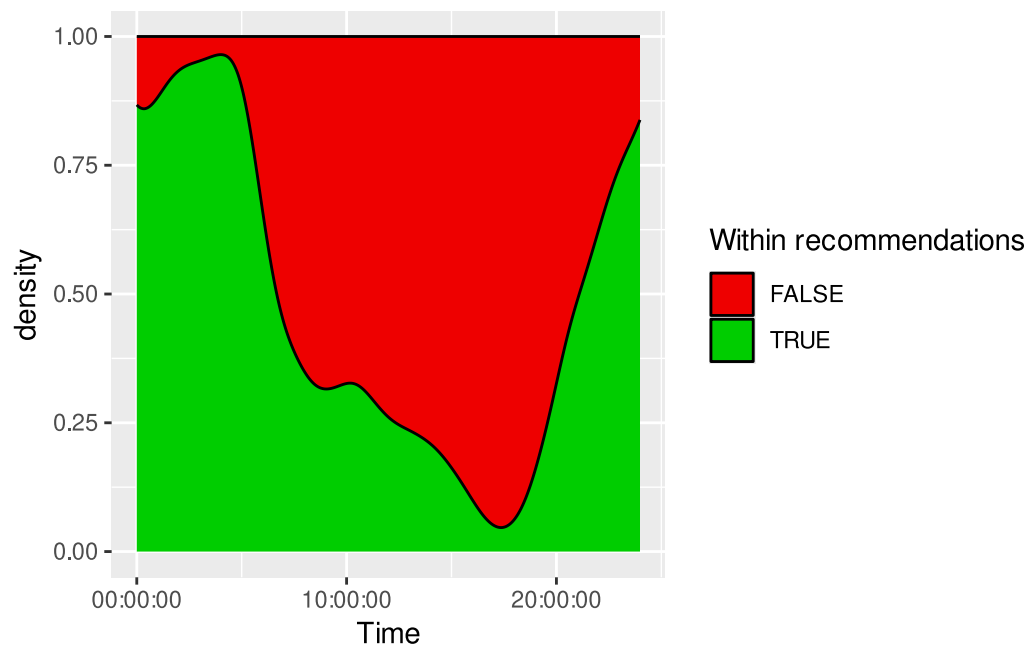
Based on the previous figure, we can add information on whether a given timepoint was adherent to the recommendations.

```
data |>
  sample_groups(3) |> #sample 3 groups
  aggregate_Datetime("15 mins", type = "floor") |> #15-minute intervals
  mutate(State.Brown = #create a factor and add an Unknown type
    factor(State.Brown |> replace_na("Unknown"),
      levels = c("wake", "pre-sleep", "sleep", "Unknown")),
    Reference.check = recode_values( #set names for adherence
      Reference.check |> as.character(),
      "TRUE" ~ "Good",
      "FALSE" ~ "Bad",
      default = "Unknown"
    ) |>
  gg_days( #create the base plot
    jco_color = FALSE, #do not use default fill scale
    geom = "ribbon", #use a ribbon geom
    aes_fill = State.Brown, #fill the ribbon by state
    group = consecutive_id(State.Brown) #group those fills by occurrences of state
  ) |>
  gg_photoperiod() |> #add photoperiod
  gg_states(Reference.check, #add state information of adherence
    aes_fill = Reference.check, #fill by adherence
    ymax = 0, alpha = 1, #only a small band
    on.top = TRUE, #put band on top
  ) +
  geom_line() + #add a line on top of everything
  labs(fill = "State") + #adjust legend label
  scale_fill_manual(values = c(wake = "skyblue3", `pre-sleep` = "gold",
    sleep = "grey", Bad = "red", Good = "green3",
    Unknown = "white")) #manual scale
```



We can also highlight when in the day recommendation is highest and lowest.

```
data |>
  add_Time_col() |> #add a time column
  drop_na(Reference.check) |> #remove instances where state is unknown
  ggplot(aes(x = Time)) + #create a plot across time
  geom_density(aes(fill = Reference.check), position = "fill") + #with scaled
  stacked densities
  scale_fill_manual(values = c("red2", "green3")) + #manual scale
  labs(fill = "Within recommendations") #adjust legend label
```



Finally, we calculate exact adherence percentages across states...

```
adherence_summary <-
data |>
  group_by(State.Brown) |> #group data by Brown state
  durations(Reference.check, #calculate the length for each group
    show.missing = TRUE, #show where data is missing
    FALSE.as.NA = TRUE) |> #regard a FALSE in the data as missing
  ungroup() |> #remove grouping
  mutate(across(duration:missing, \(x) x/total), #calculate percentages
    of.total = (total/sum(total)) |> as.numeric(), #calculate percentages
    duration = replace_values(duration, 0 ~ NA)) |> #set missing
  rename(adherence = duration,
    duration = total) |> #rename
  select(-missing) #remove unneeded column
adherence_summary
```

```
# A tibble: 4 × 4
  State.Brown adherence duration          of.total
<chr>         <dbl> <Duration>          <dbl>
1 pre-sleep    0.640 306600s (~3.55 days)  0.0986
2 sleep        0.875 957900s (~1.58 weeks) 0.308
3 wake         0.157 1416660s (~2.34 weeks) 0.455
4 <NA>         NA    429240s (~4.97 days)  0.138
```

...and add a summary row


```

adherence_summary <-
adherence_summary |>
  drop_na() |>
  summarize( #calculate summary row:
    State.Brown = "Overall",
    adherence = (adherence*of.total) |> sum(),
    duration = sum(duration) |> as.duration(),
    of.total = sum(of.total),
    adherence = adherence/of.total
  ) |>
  rbind(adherence_summary) #add the summary row to the detailed table
adherence_summary

```

```

# A tibble: 5 × 4
  State.Brown adherence duration          of.total
  <chr>         <dbl> <Duration>          <dbl>
1 Overall      0.469 2681160s (~4.43 weeks) 0.862
2 pre-sleep    0.640 306600s (~3.55 days)  0.0986
3 sleep        0.875 957900s (~1.58 weeks) 0.308
4 wake         0.157 1416660s (~2.34 weeks) 0.455
5 <NA>         NA     429240s (~4.97 days)  0.138

```

In the final step, we bring this table into a nice layout.

```

adherence_summary |>
gt() |>
  fmt_percent(c(adherence, of.total), decimals = 1) |> #format as percent
  sub_missing(missing_text = "Unkown") |> #rename missing entries
  cols_label_with(fn = \(x) str_replace(x, "\\.", " ") |> str_to_title()) |>
#transform labels
  tab_style( #show some cells bold
    cell_text(weight = "bold"),
    list(cells_column_labels(), cells_body(1))
  ) |>
  tab_style( #show a highlight for the summary row
    cell_fill("lightgrey"),
    cells_body(rows = 1)
  ) |>
  fmt_duration(duration, input_units = "seconds", output_units = "weeks") |>
#format as duration
  tab_header("Adherence to recommendations for healthy lighting", #add a header
    subtitle =
      paste0(melidos_cities[[site]], melidos_countries[[site]], sep =
        ", ")
  )

```

Adherence to recommendations for healthy lighting			
San Pedro, San JoséCosta Rica,			
State Brown	Adherence	Duration	Of Total
Overall	46.9%	4w	86.2%
pre-sleep	64.0%	<1w	9.9%
sleep	87.5%	1w	30.8%
wake	15.7%	2w	45.5%
Unkown	Unkown	<1w	13.8%

Session info

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.4 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p-r0.3.26.so;
LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=C.UTF-8      LC_NUMERIC=C           LC_TIME=C.UTF-8
 [4] LC_COLLATE=C.UTF-8    LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
 [7] LC_PAPER=C.UTF-8      LC_NAME=C              LC_ADDRESS=C
[10] LC_TELEPHONE=C        LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C

time zone: UTC
tzcode source: system (glibc)

attached base packages:
[1] stats      graphics  grDevices datasets  utils      methods    base

other attached packages:
 [1] downlit_0.4.5    xml2_1.3.8      svglite_2.2.1    gt_1.3.0
 [5] lubridate_1.9.4  forcats_1.0.0    stringr_1.6.0     dplyr_1.2.1
 [9] purrr_1.2.2      readr_2.2.0      tidyr_1.3.2       tibble_3.3.1
[13] ggplot2_4.0.3    tidyverse_2.0.0  LightLogR_0.10.3  melidosData_1.0.6

loaded via a namespace (and not attached):
 [1] gtable_0.3.6      xfun_0.57        paletteer_1.6.0    tzdb_0.5.0
 [5] vctrs_0.7.3       tools_4.6.0      generics_0.1.4     proxy_0.4-29
```

[9] pkgconfig_2.0.3	KernSmooth_2.23-26	RColorBrewer_1.1-3	S7_0.2.2
[13] lifecycle_1.0.5	compiler_4.6.0	farver_2.1.2	suntools_1.0.1
[17] textshaping_1.0.1	fontawesome_0.5.3	ggsci_3.2.0	litedown_0.7
[21] class_7.3-23	htmltools_0.5.9	sass_0.4.10	yaml_2.3.12
[25] pillar_1.11.1	gtExtras_0.6.0	classInt_0.4-11	cachem_1.1.0
[29] commonmark_2.0.0	tidyselect_1.2.1	digest_0.6.39	stringi_1.8.7
[33] rematch2_2.1.2	sf_1.0-21	labeling_0.4.3	cowplot_1.2.0
[37] fastmap_1.2.0	grid_4.6.0	cli_3.6.6	magrittr_2.0.5
[41] base64enc_0.1-6	utf8_1.2.6	el071_1.7-16	withr_3.0.2
[45] scales_1.4.0	warp_0.2.3	timechange_0.4.0	slider_0.3.2
[49] rmarkdown_2.29	otel_0.2.0	ggtext_0.1.2	ragg_1.5.0
[53] hms_1.1.3	memoise_2.0.1	evaluate_1.0.4	knitr_1.50
[57] markdown_2.0	rlang_1.2.0	gridtext_0.1.5	Rcpp_1.1.1-1.1
[61] glue_1.8.1	DBI_1.3.0	renv_1.2.3	jsonlite_2.0.0
[65] R6_2.6.1	units_0.8-7	systemfonts_1.3.1	fs_1.6.6